TASK 11 – IMPLEMENT, TEST AND DEFEND THE DESIGN

Programming in Java – Assignment 1 (CO1)

1. Introduction

The Campus Smart-Parking System is implemented using object-oriented programming principles in Java. It manages vehicle entry, parking-slot allocation, ticket generation, vehicle exit and parking-fee calculation. The implementation supports two-wheelers, cars and electric cars. Students and staff with valid monthly passes are exempt from parking fees, while visitors pay the applicable fee.

2. System Design

Class	Responsibility
Vehicle	Abstract parent class containing common vehicle information.
TwoWheeler, Car, ElectricCar	Represent vehicle categories and specialise hourly rates or charging behaviour.
ParkingUser	Stores user information and monthly-pass eligibility.
ParkingTicket	Stores ticket, vehicle, slot and duration information.
ParkingSlot	Represents one parking slot and its occupancy state.
ParkingFacility	Allocates and releases slots, manages active tickets and calculates collection.
ParkingSystemDemo	Creates test cases and executes the application.

3. Assumptions and Pricing Rules

Vehicle / user	Pricing rule
Two-wheeler	Rs. 10 per started hour
Car	Rs. 30 per started hour
Electric car	Rs. 30 per started hour + Rs. 12 per kWh
Monthly-pass holder	No parking fee
Visitor	Applicable vehicle parking fee

Every started hour is charged in full. The implementation uses integer arithmetic to calculate billable hours as (minutes + 59) / 60. Zero minutes therefore produces zero billable hours.

4. Testing and Validation

Test case	Expected result	Program result
Two-wheeler, 60 minutes	Rs. 10	Rs. 10
Car, 120 minutes	Rs. 60	Rs. 60
EV, 60 minutes + 5 kWh	Rs. 90	Rs. 90
Staff monthly pass	Rs. 0	Rs. 0
Duplicate registration	Rejected	Rejected
Zero-minute parking	Rs. 0	Rs. 0
Exactly 60 minutes	Rs. 30	Rs. 30
Invalid floor / slot	Rejected	Rejected
Unknown vehicle type	Rejected	Rejected
Last available slot	Allocated	Allocated

5. Manual Daily Collection Calculation

Vehicle	Calculation	Fee
Two-wheeler	1 × Rs. 10	Rs. 10
Car	2 × Rs. 30	Rs. 60
Electric car	(1 × Rs. 30) + (5 × Rs. 12)	Rs. 90
Staff monthly pass	Exempt	Rs. 0
Total	**10 + 60 + 90 + 0**	**Rs. 160**

The manually calculated collection is Rs. 160, which matches the program's displayed total collection.

6. Alternative Data Structures and Complexity

Operation	Array	ArrayList	HashMap
Find by registration	O(n)	O(n)	O(1) average
Add ticket	O(1) if space	O(1) amortised	O(1) average
Remove ticket	O(n)	O(n)	O(1) average
Storage	O(n)	O(n)	O(n)

The active-ticket array uses sequential search, requiring O(n) time to find a vehicle. An ArrayList has similar lookup complexity. A HashMap keyed by registration number provides O(1) average lookup and removal, but adds hashing and map-entry overhead. The array is simple and suitable for a small facility with fixed capacity; a HashMap is preferable when faster lookup is important.

7. Evaluation and Limitations

The design uses encapsulation to protect data, inheritance to reuse common vehicle properties, overriding to specialise vehicle rates, and polymorphism to handle different vehicle types through a common Vehicle reference. A jagged array supports floors with different slot counts. The current design uses fixed pricing and an in-memory array; dynamic pricing, reservations, online payments and persistent storage would require further development.

8. Conclusion

The Campus Smart-Parking System has been implemented and tested for normal operations, boundary conditions and invalid cases. The manually calculated daily collection matches the program result of Rs. 160. The array-based design is straightforward for a small fixed-capacity facility, while a HashMap would improve average ticket lookup performance as the system grows.